

C++

INHERITANCE

A Complete Guide to Object-Oriented Reusability & Hierarchy

Topics: Basics • Types • Access Specifiers

TABLE OF CONTENTS

01

What is Inheritance?

Real-world analogies & motivation

02

Types of Inheritance

Single, Multiple, Multilevel, Hierarchical, Hybrid

03

Access Specifiers

public, private, protected inheritance

04

Base & Derived Classes

Syntax, constructors, destructors

05

Function Overriding

Redefining base class methods

06

Virtual Functions

Runtime polymorphism

07

Abstract Classes

Pure virtual functions & interfaces

08

Real-Life Examples

Employee, Vehicle, Shape hierarchies

09

Best Practices

Design patterns & pitfalls

REUSABILITY?

- Reusability means that a class that has been designed, created and debugged can be distributed to other programmers for use in their own programs.
 - Similar to the idea of a library of functions in a procedural language.
- The concept of inheritance provides an important extension to the idea of reusability, as it allows a programmer to **take an existing class without modifying it and adding additional features and functionality**. This is done by inheriting a new sub-class from the existing base class.

SECTION 01

What is Inheritance?

Concepts • Terminology • Motivation • Real-World Analogies

WHAT IS INHERITANCE?

Inheritance is the mechanism in C++ by which one class (the **derived class**) acquires the properties and behaviors of another class (the **base class**). It enables **code reuse**, **hierarchical classification**, and forms the backbone of **polymorphism**.

1 Code Reuse

Write once, use in many derived classes. Avoid duplication.

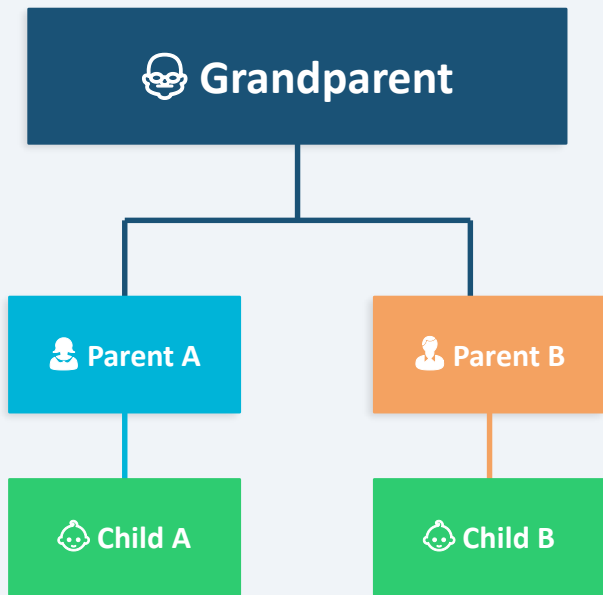
2 Extensibility

Extend existing classes with new features without modifying them.

3 Hierarchy

Model real-world 'is-a' relationships in a logical tree.

REAL-LIFE ANALOGY: BIOLOGICAL INHERITANCE



Biological Family Tree

How Biology Maps to C++ Inheritance

Grandparent class

→ *Base class (root of hierarchy)*

Parent inherits from Grandparent

→ *Derived class extends Base class*

Child inherits from Parent

→ *Sub-derived inherits from derived*

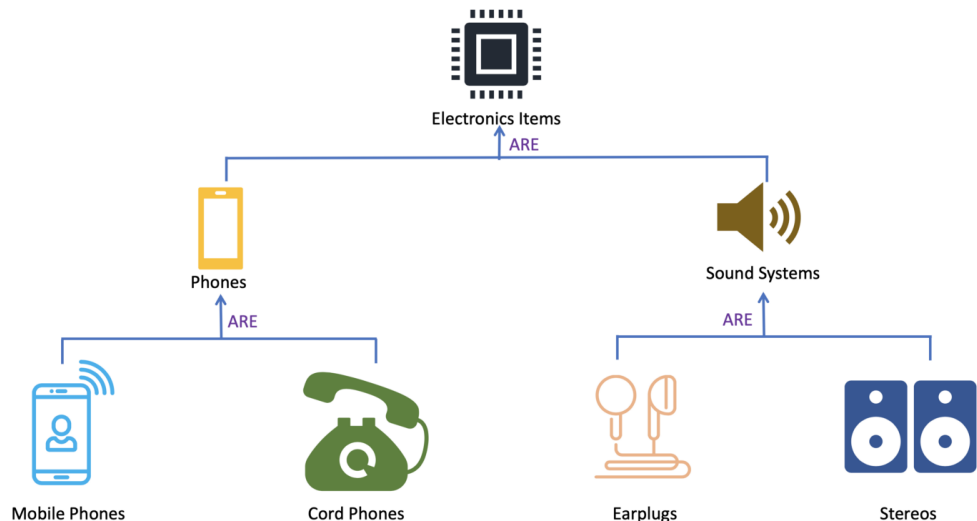
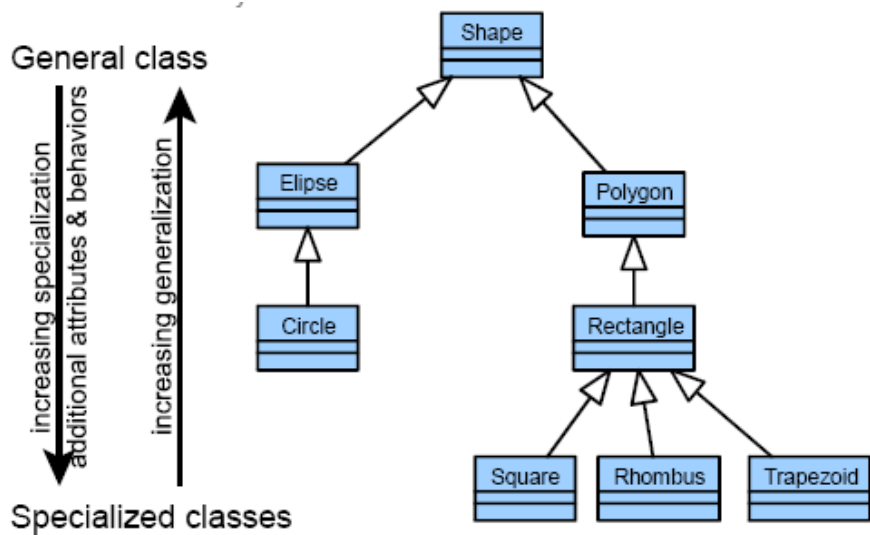
Traits passed down (eye color, height)

→ *Data members & methods inherited*

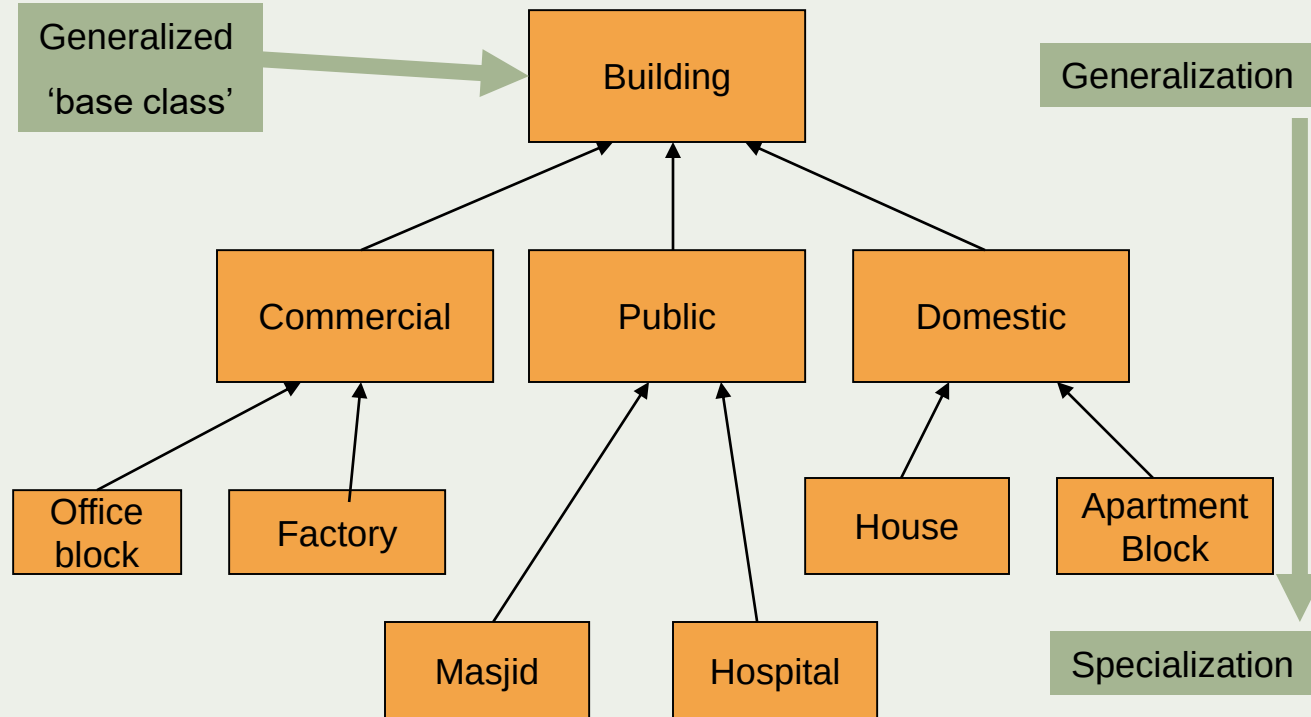
Children can have unique features

→ *Derived class adds new members*

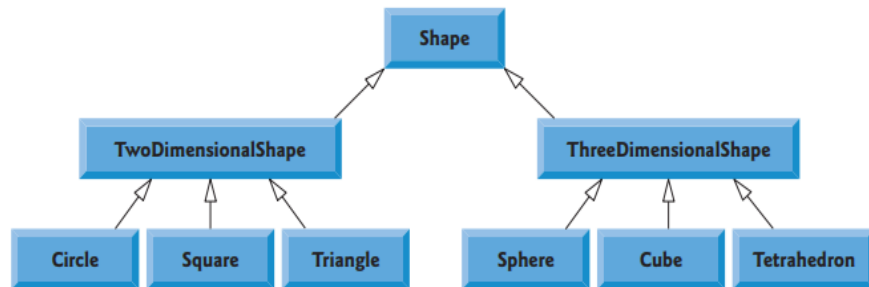
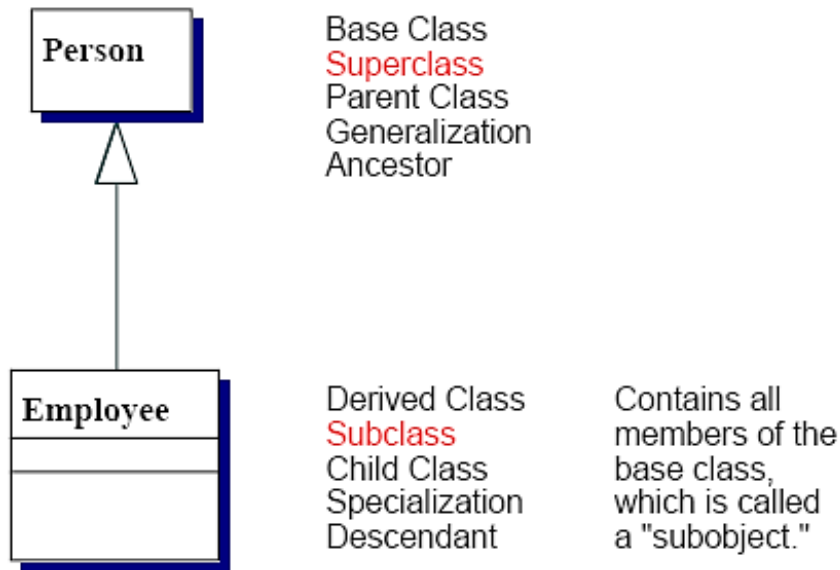
Examples



Example

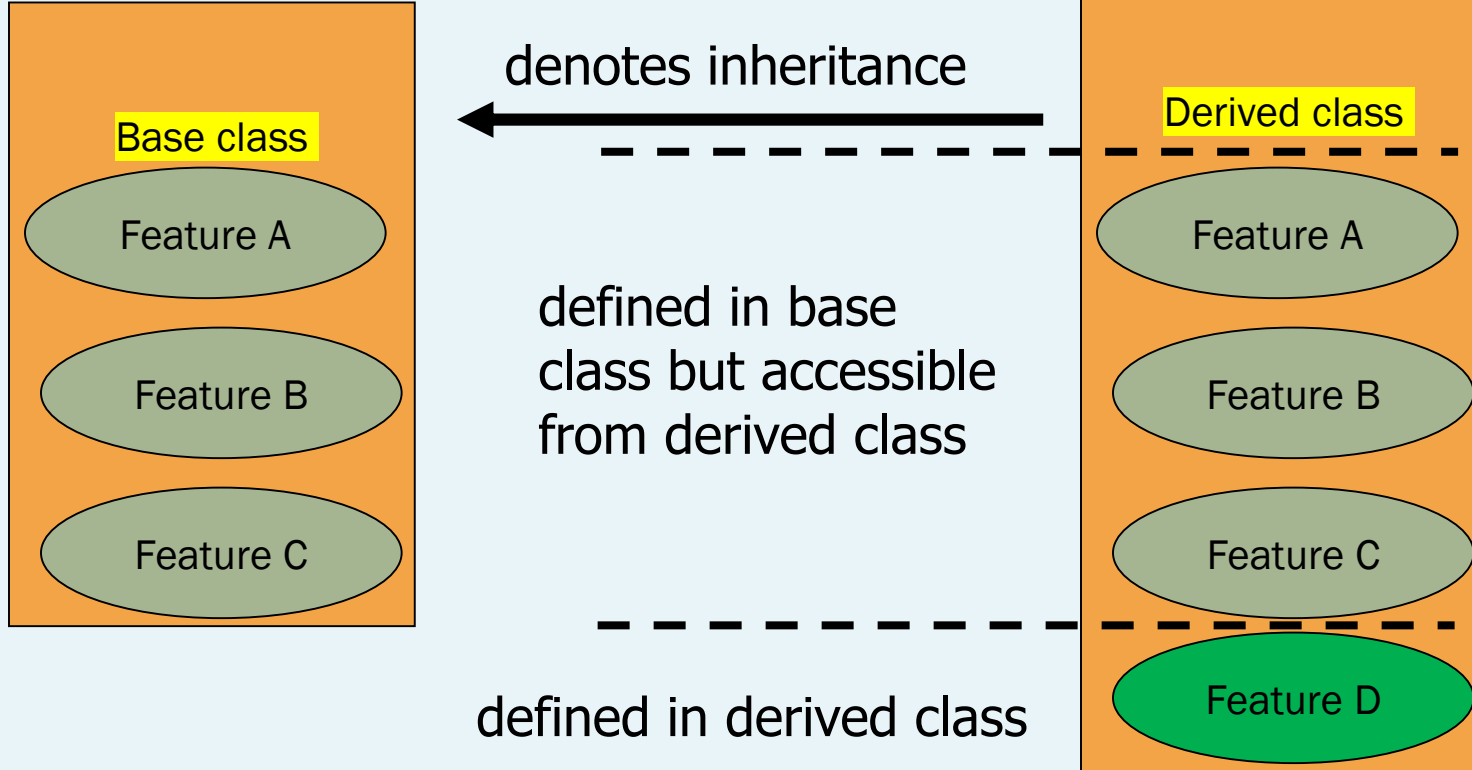


Examples



- Every object of the derived class is also object of the base class but Objects of the base class are not objects of the derived class
 - All cars are vehicles, but not all vehicles are cars
- The parent is a generalization; the child is a specialization

Inheritance Example



KEY TERMINOLOGY

Base Class

Parent Class / Superclass

The existing class whose properties are inherited.
Defines common attributes and behaviors.

Derived Class

Child Class / Subclass

The new class that inherits from the base class and
can extend/override its functionality.

'is-a' Relationship

Inheritance test

A Car 'is-a' Vehicle. This relationship validates
whether inheritance is appropriate.

'has-a' Relationship

Composition/Aggregation

A Car 'has-a' Engine. This indicates composition, NOT
inheritance.

KEY TERMINOLOGY

Ancestor / Descendant

Root / Leaf in hierarchy

A class's ancestors are all superclasses up the chain; descendants are all subclasses below.

Method Overriding

Function Redefinition

Derived class redefines a base class method with the same signature and return type.

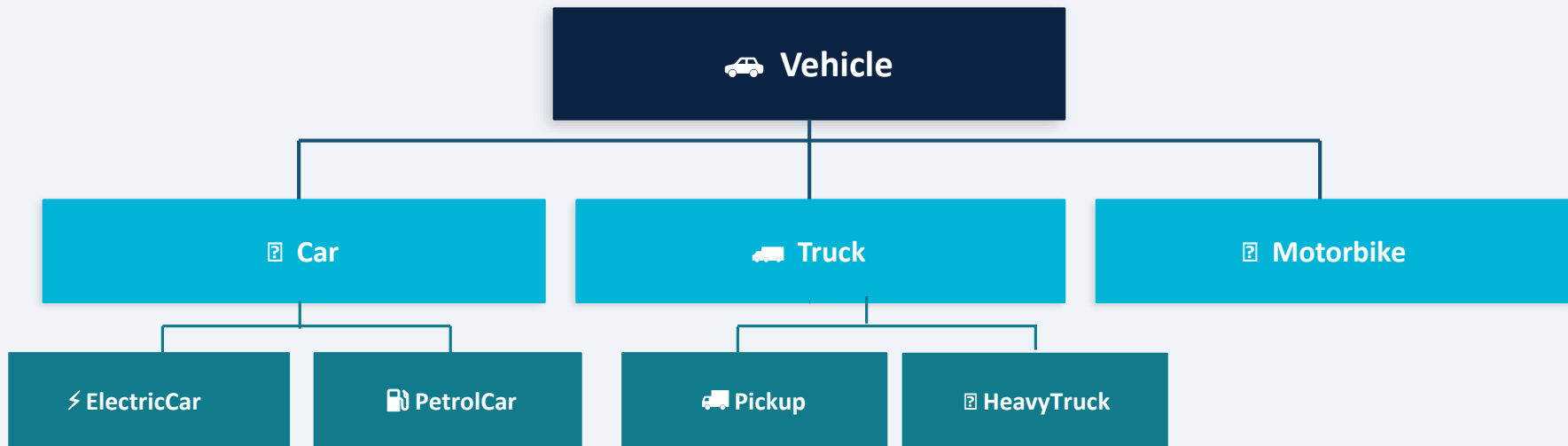
Ancestor

A class's ancestors are those from which its own super classes inherit

Descendant

A class's descendants are those which inherit from its sub classes.

CLASSIFICATION HIERARCHY: REAL-WORLD EXAMPLE

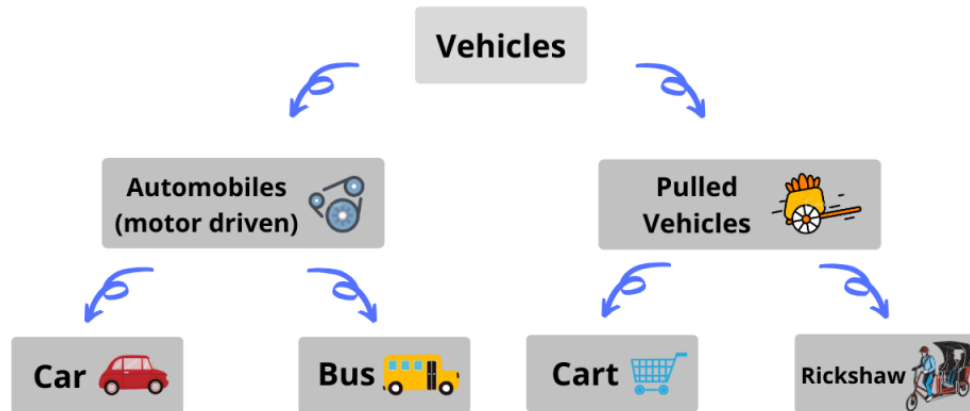


Key Principle:

Every subclass node satisfies the **'is-a' test** with its parent. An ElectricCar **is-a** Car, which **is-a** Vehicle. Inheritance should only be used when this relationship holds.

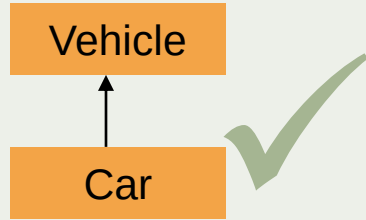
INHERITANCE

- A direct base class is inherited explicitly (one level up hierarchy)
- An indirect base class is inherited two or more levels up hierarchy
- Every object of the derived class is also object of the base class
- Objects of the base class are not objects of the derived class

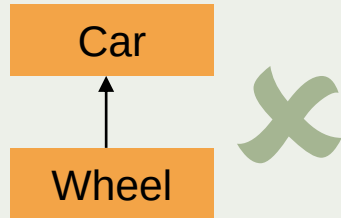


Designing your classification hierarchy:

'A kind of'/'is-A' or 'a part of'/'has-A'?



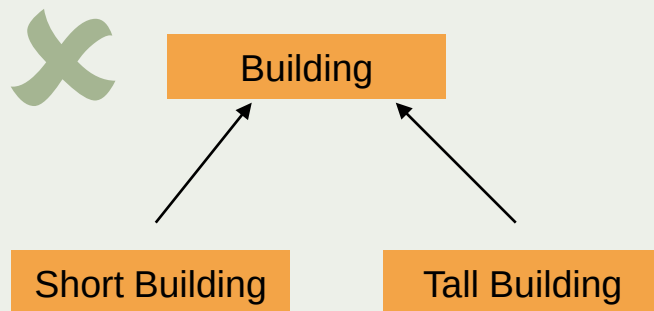
A car is 'a kind of' vehicle
car class can inherit from
vehicle class



A wheel isn't 'a kind of' car.
A wheel is 'a part of' a car

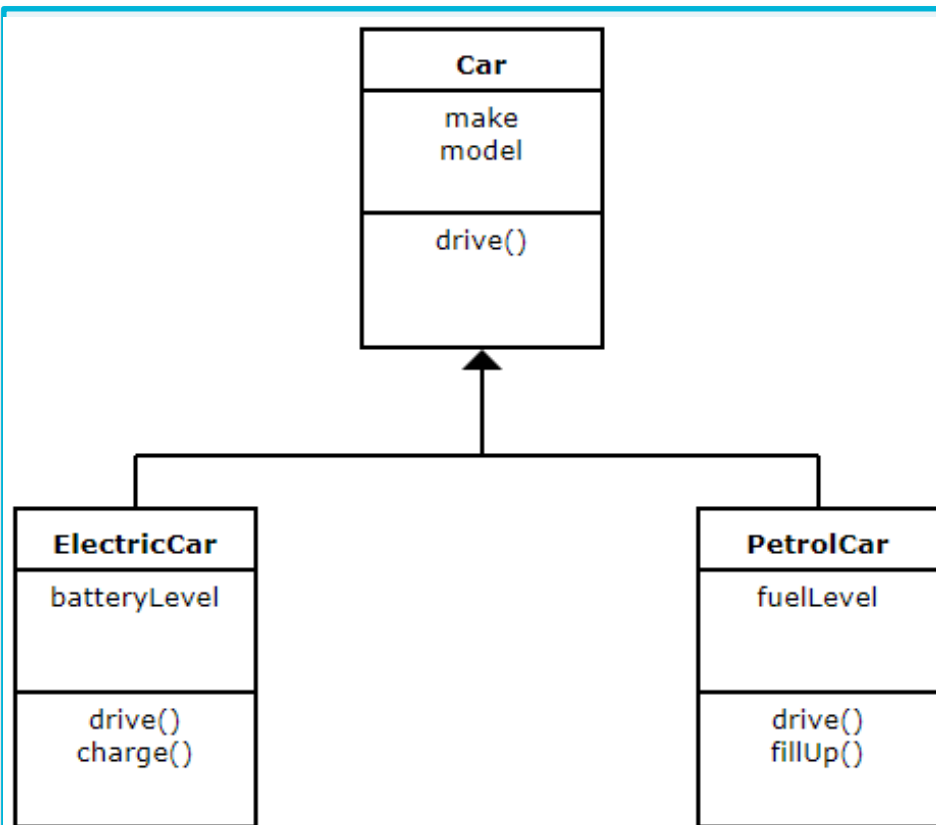
Designing your classification hierarchy: Different classes or different states?

- Need to analyze whether differences between objects are dependant on type (such as a house being different to a factory) or state (different values of the class's attributes)



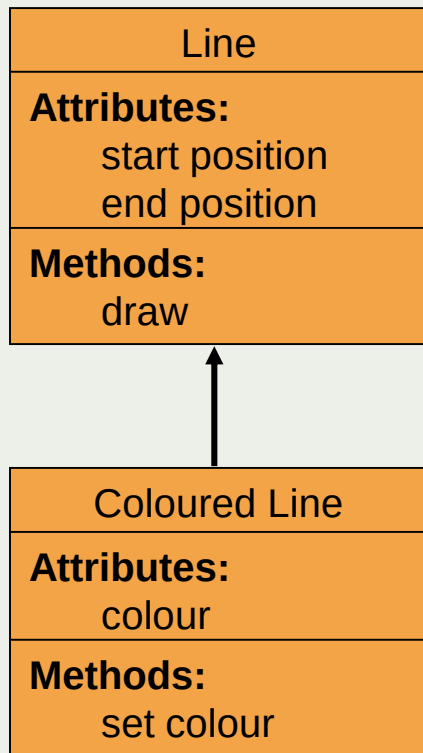
short building and tall building might vary only in the value of the height attribute - don't need separate classes

Example



A sub-class shares common methods with its super-class but can add its own methods or overwrite the methods of its super-class.

What do objects inherit?



- A 'coloured line' is a kind of line
- The coloured line *class* inherits all the attributes and methods of the line class and adds attributes and methods of its own

An *object* of the 'coloured line' class has all the attributes and methods of the 'line' base class as well as the attributes and methods added by the derived class

Specialization

- Extending the functionality of an existing class
 - eg a coloured line is a specialised kind of line
- A class is both
 - *closed* in that it has an encapsulated, private part which cannot be affected by external manipulation
 - *open* in that it allows itself to be used as part of a larger software unit.

Generalization

- Sharing commonality between two or more classes
- If we were modelling animals in a zoo would we create a separate class for each animal type?

Cow

Whale

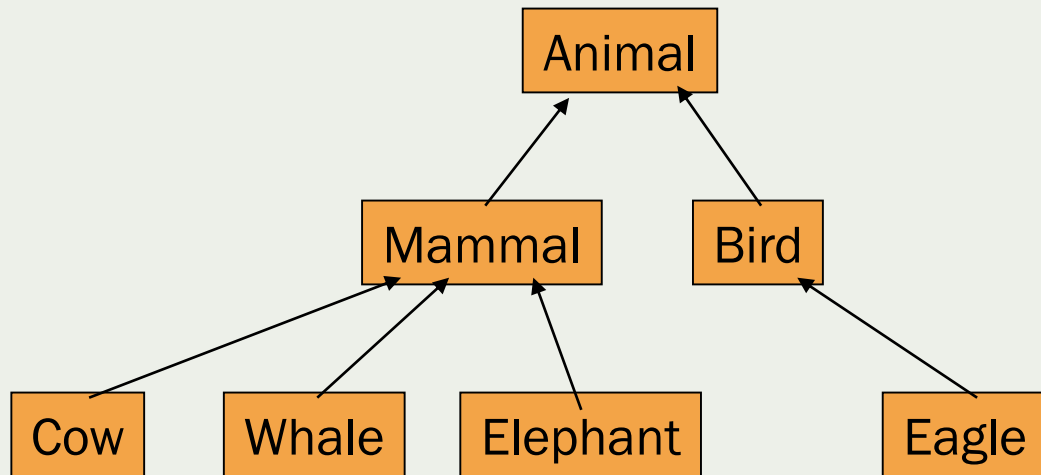
Elephant

Eagle

- This would duplicate attributes such as legs and methods such as `getAge()`

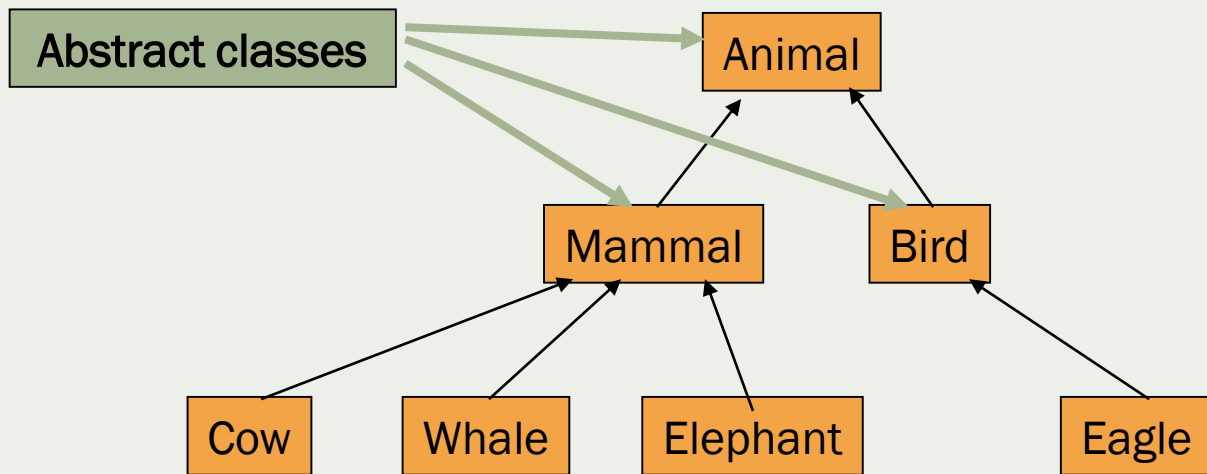
Generalization

- Helpful to place common elements (attributes and methods) in a shared base class and organize problem into an inheritance hierarchy.



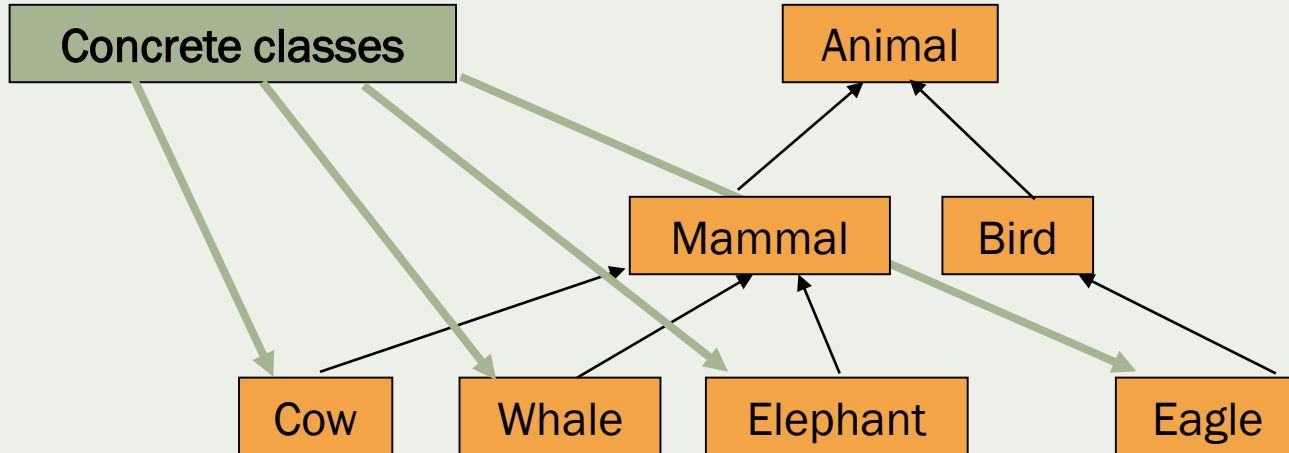
Generalization

- Sometimes this leads to the creation of abstract classes which can't be instantiated directly



Generalization

- Concrete classes can be instantiated directly



Inheritance Examples – Based & Derived Classes

Base class	Derived classes
Student	GraduateStudent, UndergraduateStudent
Shape	Circle, Triangle, Rectangle, Sphere, Cube
Loan	CarLoan, HomeImprovementLoan, MortgageLoan
Employee	Faculty, Staff
Account	CheckingAccount, SavingsAccount

INHERITANCE SYNTAX IN C++

C++ Example 1

// Base class

```
class Vehicle {
protected:
    string brand;
    int speed;
public:
    Vehicle(string b, int s)
        : brand(b), speed(s) {}

    void display() {
        cout << "Brand: " << brand;
        cout << " Speed: " << speed;
    }
};
```

// Derived class

```
class Car : public Vehicle {
private:
    int doors;
public:
    Car(string b, int s, int d)
        : Vehicle(b, s), doors(d)
    {}

    void showDoors() {
        cout << "Doors: " << doors;
    }
};

// Usage
int main() {
    Car c("Toyota", 180, 4);
    c.display(); // inherited!
    c.showDoors();// own method
}
```

→ Syntax: `class Derived : accessSpecifier Base { };`

Access control of the inherited members:

- **public** (most common)
- **private** (default, but little used)
- **protected**

→ *protected members accessible in derived*

→ *Base constructor called via initializer list*

→ *Derived object uses inherited method*

Inheritance Example 2

Determining which method is called

```
class shape {  
private:  
    int color;  
public:  
    shape(int c) : color(c)  
    { }  
    void draw( ) { ... }  
    void setColor(int C) {  
        color = C;  
    }  
};
```

```
shape S(blue);  
circle C(25, yellow);  
S.draw( );           // runs Shape draw  
C.draw( );           // runs Circle draw  
C.setColor(red);     // runs Shape setColor
```

```
class circle : public shape {  
private:  
    int radius;  
public:  
    circle(int r, int c) : shape(c), radius(r)  
    { }  
    void draw ( ) {  
        shape::draw( );  
    }  
};
```

Because a circle is a shape, C has a “color” attribute and setColor function.

Inheritance in the English Distance Class (1/3)

```
// inheritance using English Distances
#include <iostream>
using namespace std;
enum posneg { pos, neg };    // for sign in DistSign
class Distance{              // English Distance class
protected:                  // NOTE: can't be private
    int feet; float inches;
public:
    Distance() : feet(0), inches(0.0) { } // no-arg constructor
    Distance(int ft, float in) : feet(ft), inches(in) { }
    void getdist(){           // get length from user
        cout << "\nEnter feet: " ; cin >> feet;
        cout << "Enter inches: " ; cin >> inches;
    }
    void showdist() const      // display distance
    { cout << feet << "\'-" << inches << '\\"'; }
};
```

Inheritance in the English Distance Class (2/3)

```
class DistSign : public Distance{           // adds sign to Distance
private: posneg sign;                       // sign is pos or neg
public:                                     // no-arg constructor
    DistSign() : Distance()                // call base constructor
    { sign = pos; }                       // set the sign to +

    DistSign(int ft, float in, posneg sg=pos): // 2- or 3-arg constructor
    Distance(ft, in)                        // call base constructor
    { sign = sg; }                          // set the sign

    void getdist(){                         // get length from user
        Distance::getdist();               // call base getdist()
        char ch;                           // get sign from user
        cout << "Enter sign (+ or -): "; cin >> ch;
        sign = (ch=='+') ? pos : neg;}

    void showdist() const{                  // display distance
        cout << ( (sign==pos) ? "(+)" : "(-)" ); // show sign
```

Inheritance in the English Distance Class (3/3)

```
        Distance::showdist();                // ft and in
    }
};

int main(){
    DistSign alpha;                          // no-arg constructor
    alpha.getdist();                          // get alpha from user
    DistSign beta(11, 6.25);                  // 3-arg constructor
    DistSign gamma(100, 5.5, neg);            // 3-arg constructor
                                            // display all distances

    cout << "\nalpha = "; alpha.showdist();
    cout << "\nbeta  = "; beta.showdist();
    cout << "\ngamma = "; gamma.showdist();
    cout << endl;
    return 0;
}
```

```
Enter feet: 4
Enter inches: 3.25
Enter sign (+ or -): -
```

```
alpha = (-)4'-3.25"
beta  = (+)11'-6.25"
gamma = (-)100'-5.5"
```

SECTION 02

Types of Inheritance

Single • Multiple • Multilevel • Hierarchical • Hybrid

5 TYPES OF INHERITANCE IN C++

1

Single

Inheritance

$A \rightarrow B$

One derived class
from one base class

2

Multiple

Inheritance

$A, B \rightarrow C$

One derived class
from multiple base
classes

3

Multilevel

Inheritance

$A \rightarrow B \rightarrow C$

Chain of
inheritance:
 $A \rightarrow B \rightarrow C$

4

Hierarchical

Inheritance

$A \rightarrow B, C, D$

Multiple derived
classes from one
base

5

Hybrid

Inheritance

Mix of above

Combination of two
or more types

TYPE 1: SINGLE INHERITANCE

Animal

```
- name: string  
- age: int  
+ speak(): void  
+ eat(): void
```

inherits

Dog : Animal

```
// Inherits: name, age  
// Inherits: speak(), eat()  
- breed: string  
+ fetch(): void
```

EXAMPLE: Animal → Dog

```
class Animal {  
protected:  
    string name;  
    int age;  
public:  
    Animal(string n, int a)  
        : name(n), age(a) {}  
    void speak() {  
        cout << name << " makes sound";  
    }  
    void eat() {  
        cout << name << " is eating";  
    }  
};  
  
class Dog : public Animal {  
private:  
    string breed;  
public:  
    Dog(string n, int a, string b)  
        : Animal(n, a), breed(b) {}  
    void fetch() {  
        cout << name << " fetches!";  
    }  
};
```

```
int main() {  
    Dog d("Rex", 3, "Labrador");  
    d.speak(); //Inherited from Animal  
    d.eat();   //Inherited from Animal  
    d.fetch(); //Dog's own method  
}
```


TYPE 2: MULTIPLE INHERITANCE

Flyable

```
+ fly(): void  
+ altitude: int
```

Swimmable

```
+ swim(): void  
+ depth: int
```

Example

```
class A // base class A  
{  
};  
class B // base class B  
{  
};  
// C is derived from A and B  
class C : public A, public B  
{  
};
```

Duck : Flyable, Swimmable

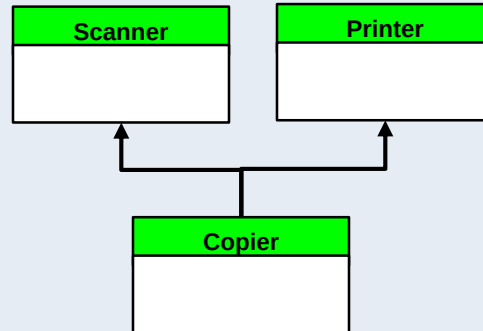
```
+ quack(): void  
// Can fly AND swim!
```

A class can be derived from more than one base class. This is called **multiple inheritance**. Figure shows how this looks when a class **Copier** is derived from base classes **Scanner** and **Printer**.

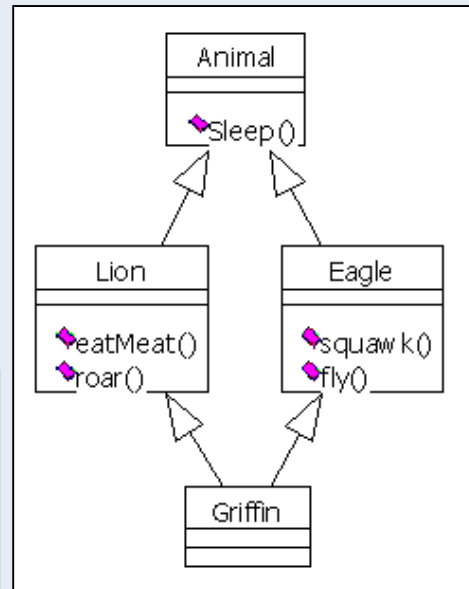
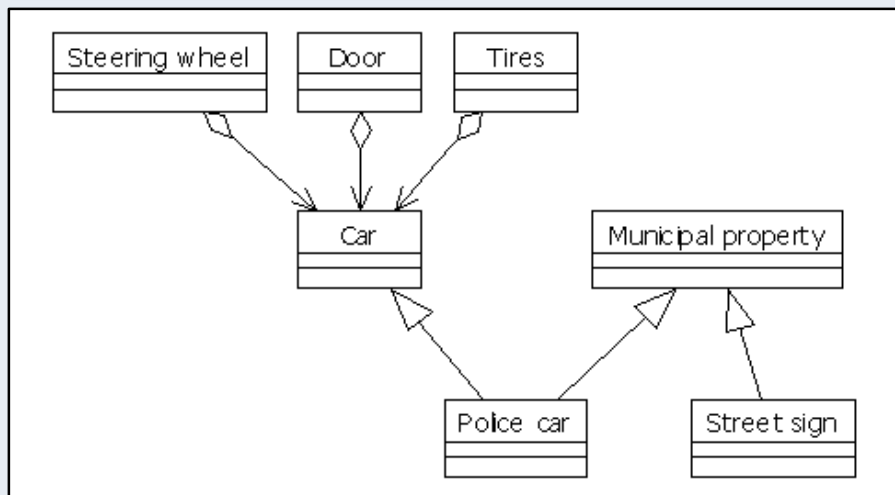
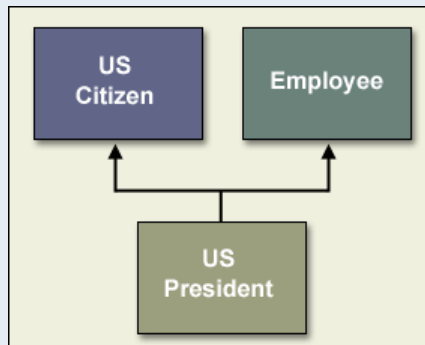
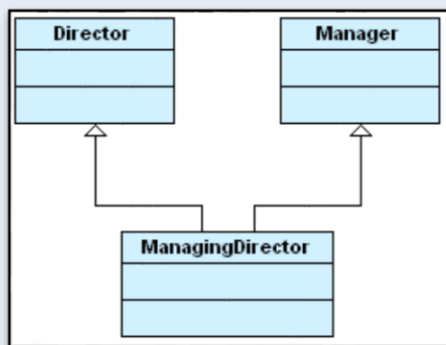
⚠️ Diamond Problem: Multiple inheritance can cause ambiguity if two base classes share a common ancestor. C++ resolves this using virtual base classes.

MULTIPLE INHERITANCE

```
class Scanner // base class Scanner
{ };
class Printer // base class Printer
{ };
class Copier : public Scanner, public Printer
{ };
// Copier is derived from Scanner and printer
```



MULTIPLE INHERITANCE



MEMBER FUNCTIONS IN MULTIPLE INHERITANCE

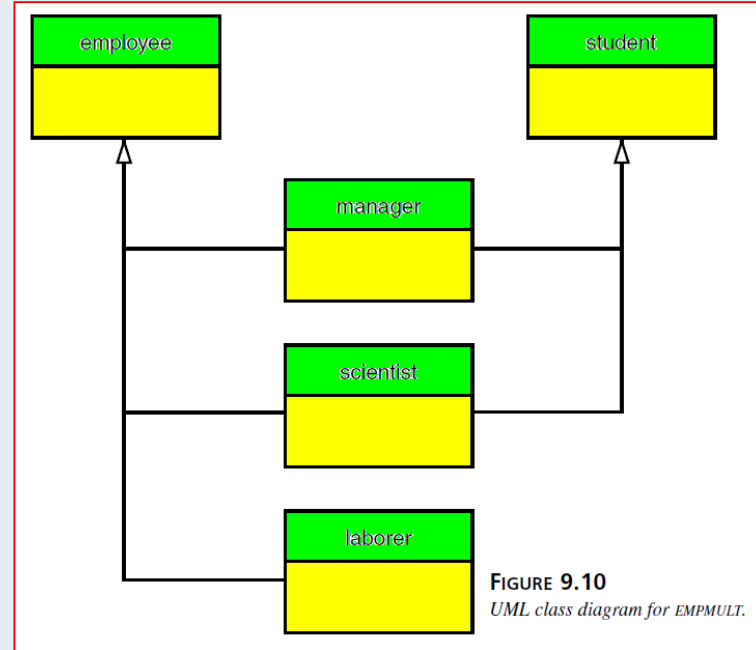
```
class student
{ };

class employee
{ };

class manager :
private employee, private
student
{ };

class scientist :
private employee, private
student
{ };

class laborer : public
employee
{ };
```



MEMBER FUNCTIONS IN MULTIPLE INHERITANCE -- 1/5

```
//multiple inheritance with employees and degrees
#include <iostream>
using namespace std;
const int LEN = 80;           // maximum length of names
class student{               // educational background
private:
    char school[LEN];         // name of school or university
    char degree[LEN];        // highest degree earned
public:
    void getedu(){
        cout << "    Enter name of school or university: ";
        cin >> school;
        cout << "    Enter highest degree earned \n";
        cout << "    (Highschool, Bachelor's, Master's, PhD): ";
        cin >> degree;
    }
}
```

MEMBER FUNCTIONS IN MULTIPLE INHERITANCE -- 2/5

```
void putedu() const{
    cout << "\n   School or university: " << school;
    cout << "\n   Highest degree earned: " << degree;
}
};
class employee{
private:
    char name[LEN];                // employee name
    unsigned long number;          // employee number
public:
    void getdata(){
        cout << "\n   Enter last name: "; cin >> name;
        cout << "   Enter number: ";      cin >> number;
    }
    void putdata() const{
        cout << "\n   Name: " << name;
        cout << "\n   Number: " << number;
```

MEMBER FUNCTIONS IN MULTIPLE INHERITANCE -- 3/5

```
    }  
};  
class manager : private employee, private student{  
private:  
    char title[LEN];                // "vice-president" etc.  
    double dues;                    // golf club dues  
public:  
    void getdata(){  
        employee::getdata();  
        cout << "    Enter title: ";      cin >> title;  
        cout << "    Enter golf club dues: "; cin >> dues;  
        student::getedu();  
    }  
    void putdata() const{  
        employee::putdata();  
        cout << "\n    Title: " << title;  
        cout << "\n    Golf club dues: " << dues;
```

MEMBER FUNCTIONS IN MULTIPLE INHERITANCE -- 4/5

```
        student::putedu();
    }
};

class scientist : private employee, private student{
private: int pubs;           // number of publications
public:
    void getdata(){
        employee::getdata();
        cout << "    Enter number of pubs: "; cin >> pubs;
        student::getedu();
    }
    void putdata() const{
        employee::putdata();
        cout << "\n    Number of publications: " << pubs;
        student::putedu();
    }
};
```


MEMBER FUNCTIONS IN MULTIPLE INHERITANCE -- 5/5 xc

```
class laborer : public employee { };

int main(){
    manager m1;      scientist s1, s2;      laborer l1;
    cout << endl;
    cout << "\nEnter data for manager 1"; // get data for
    m1.getdata();                          //several employees
    cout << "\nEnter data for scientist 1"; s1.getdata();
    cout << "\nEnter data for scientist 2"; s2.getdata();
    cout << "\nEnter data for laborer 1";   l1.getdata();
    cout << "\nData on manager 1";          //display data for
    m1.putdata();                          //several employees
    cout << "\nData on scientist 1";        s1.putdata();
    cout << "\nData on scientist 2";        s2.putdata();
    cout << "\nData on laborer 1";         l1.putdata();
    cout << endl;
    return 0; }
```

Ambiguity in Multiple Inheritance

- Two base classes have functions with the same name, while a class derived from both base classes has no function with this name.
 - How do objects of the derived class access the correct base class function?
 - The name of the function alone is insufficient, since the compiler can't figure out which of the two functions is meant.
 - Next program demonstrates this situation.
- Another kind of ambiguity arises if you derive a class from two classes that are each derived from the same class.
 - This creates a **diamond-shaped inheritance** tree.
 - The DIAMOND program shows how this looks

Ambiguity in Multiple Inheritance

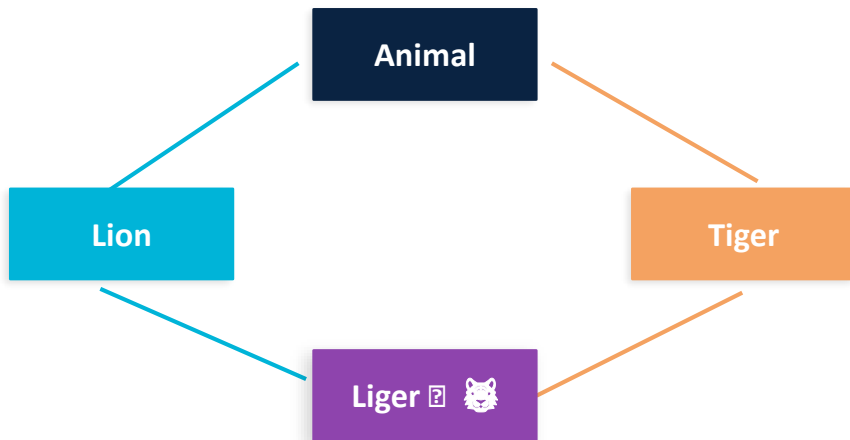
```
// demonstrates ambiguity in multiple inheritance
#include <iostream>
using namespace std;
class A{
    public: void show() { cout << "Class A\n"; }
};
class B{
    public: void show() { cout << "Class B\n"; }
};
class C : public A, public B { };
int main() {
    C objC;           // object of class C
    // objC.show(); //ambiguous--will not compile
    objC.A::show(); //OK
    objC.B::show(); //OK
    return 0;
}
```

Ambiguity in Multiple Inheritance – Diamond Problem

```
//diamond.cpp investigates diamond-shaped multiple inheritance
#include <iostream>
using namespace std;
class A { public: void func() {cout << "Hello World \n" ; } };
class B : public A { };
class C : public A { };
class D : public B, public C { };
int main(){
    D objD;
    objD.func();    // ambiguous: won't compile
    return 0;
}
```

```
error C2385: 'D::func' is ambiguous
warning C4385: could be the 'func' in base 'A' of base 'B' of class 'D'
warning C4385: or the 'func' in base 'A' of base 'C' of class 'D'
```

THE DIAMOND PROBLEM & VIRTUAL INHERITANCE

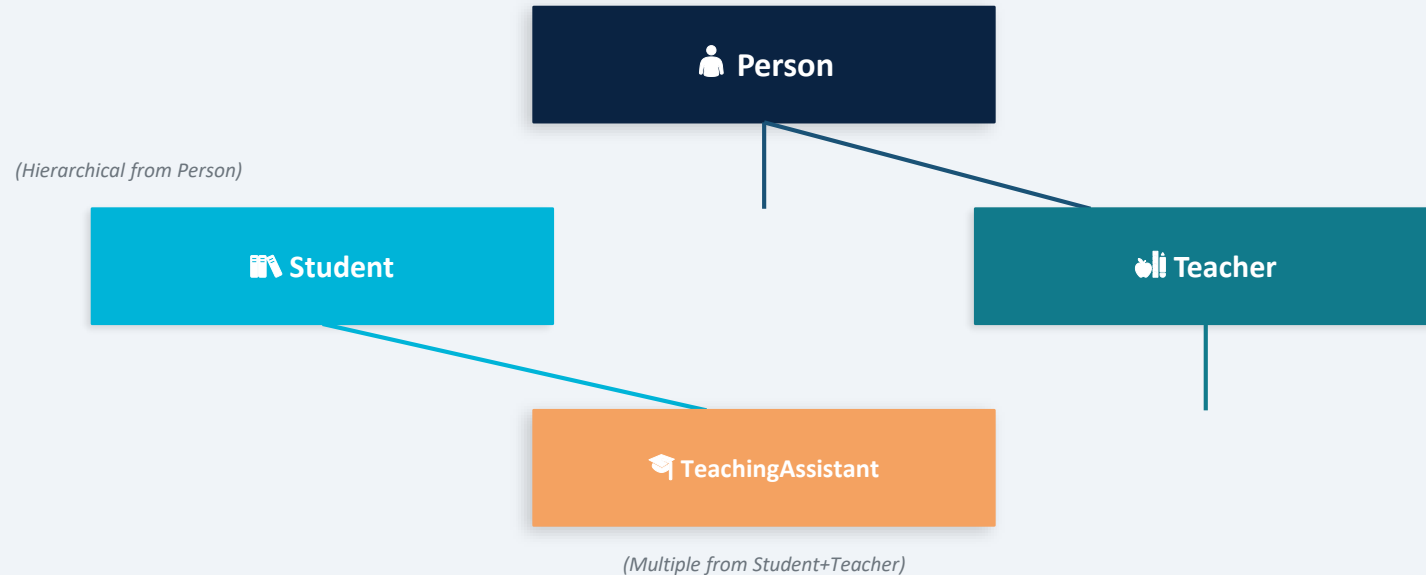


```
// Problem: Liger gets TWO copies of Animal!
class Animal { public: int age; };
class Lion : public Animal {}; // Lion has Animal::age
class Tiger : public Animal {}; // Tiger has Animal::age
class Liger : public Lion, public Tiger {};
// TWO Animal::age!
// liger.age = 5; // ✗ AMBIGUOUS! Which age?

// Fix: Virtual Inheritance
class Lion : virtual public Animal {}; // Only ONE Animal
copy
```

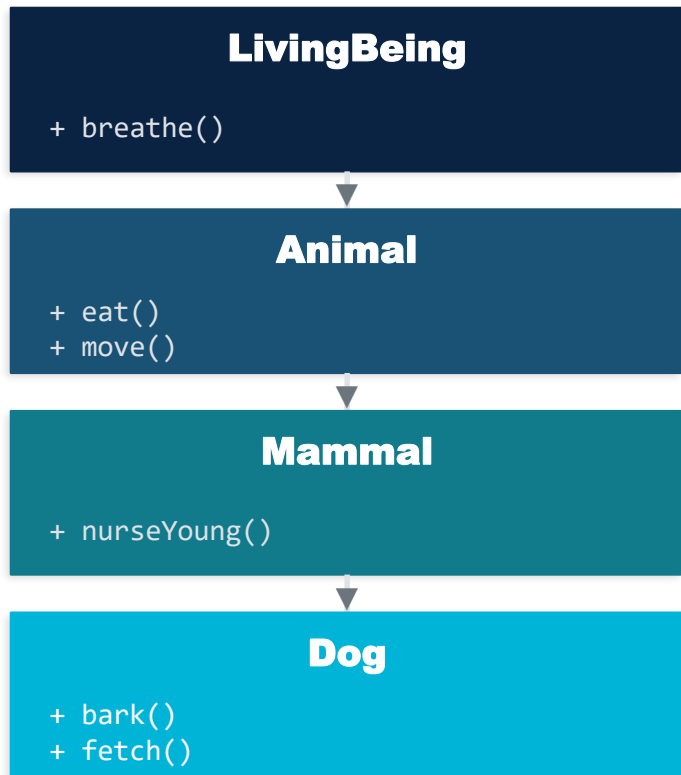
```
class Tiger : virtual public Animal {}; // shared by both
class Liger : public Lion, public Tiger {};
// liger.age = 5; // ✔ OK – only one Animal subobject
```

TYPE 5: HYBRID INHERITANCE — University System



```
class Person { protected: string name; };
class Student : virtual public Person { /* enrollment, grades */ }; // virtual!
class Teacher : virtual public Person { /* courses, salary */ }; // virtual!
class TeachingAssistant : public Student, public Teacher { // Multiple!
    // TA is BOTH Student AND Teacher
    // virtual inheritance ensures only ONE Person subobject
};
```

TYPE 3: MULTILEVEL INHERITANCE



MULTILEVEL EXAMPLE

```
class LivingBeing {
public:
    void breathe() {
        cout<<"Breathing"; }
};

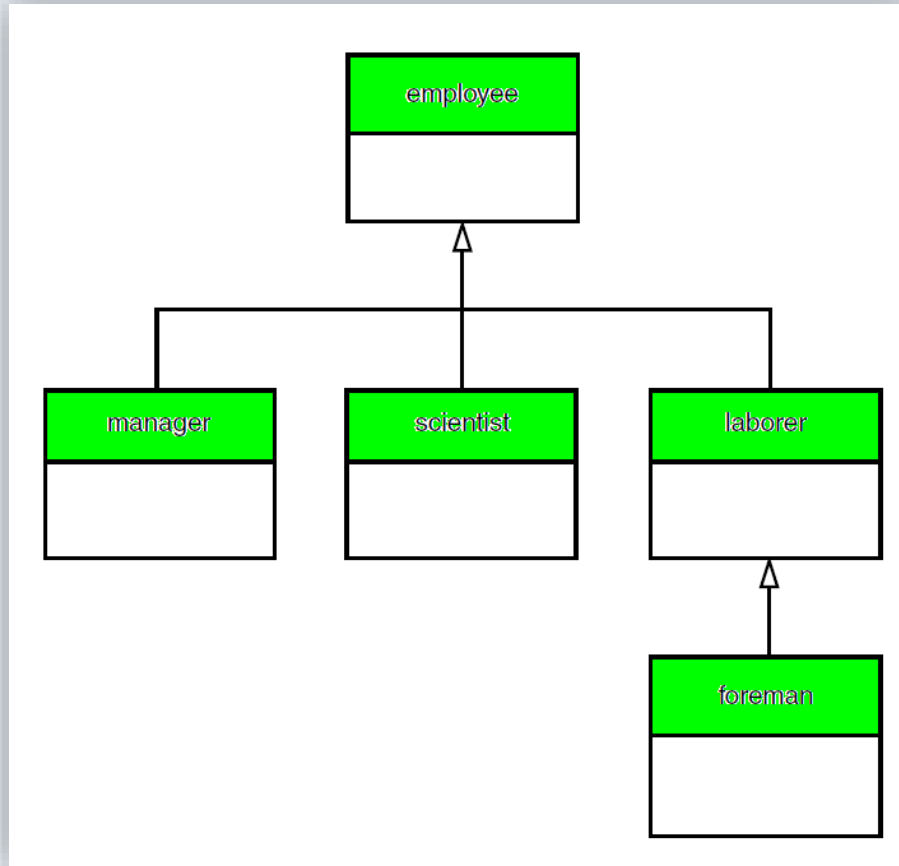
class Animal : public LivingBeing {
public:
    void eat() { cout<<"Eating"; }
    void move() { cout<<"Moving"; }
};

class Mammal : public Animal {
public:
    void nurseYoung() {
        cout<<"Nursing young";
    }
};

class Dog : public Mammal {
public:
    void bark() { cout<<"Woof!"; }
    void fetch() {
        cout<<"Fetching!"; }
};

// Dog inherits ALL methods:
// breathe(), eat(), move(),
// nurseYoung(), bark(), fetch()
int main() {
    Dog d;
    d.breathe();    // from
                    // LivingBeing
    d.eat();        // from Animal
    d.nurseYoung(); // from Mammal
    d.bark();       // Dog's own
}
```

MULTILEVEL INHERITANCE -- EXAMPLE



MULTILEVEL INHERITANCE – EXAMPLE [1/4]

```
// multiple levels of inheritance
#include <iostream>
#include <stdio.h>    // for fflush()
using namespace std;
const int LEN = 80;
class employee{
private:
    char name[LEN]; unsigned long number;
public:
    void getdata(){
        cout << "\n  Enter last name: "; cin.get(name,LEN);
        cout << "    Enter number: ";
        cin >> number; fflush (stdin);}
    void putdata() const{
        cout << "\n  Name: " << name;
        cout << "\n  Number: " << number;
    } };
};
```

MULTILEVEL INHERITANCE – EXAMPLE [2/4]

```
class scientist : public employee // scientist class
{
private:
    int pubs; // number of publications
public:
    void getdata()
    {
        employee::getdata();
        cout << "    Enter number of pubs: "; cin >> pubs;
    }
    void putdata() const
    {
        employee::putdata();
        cout << "\n    Number of publications: " << pubs;
    }
};
```

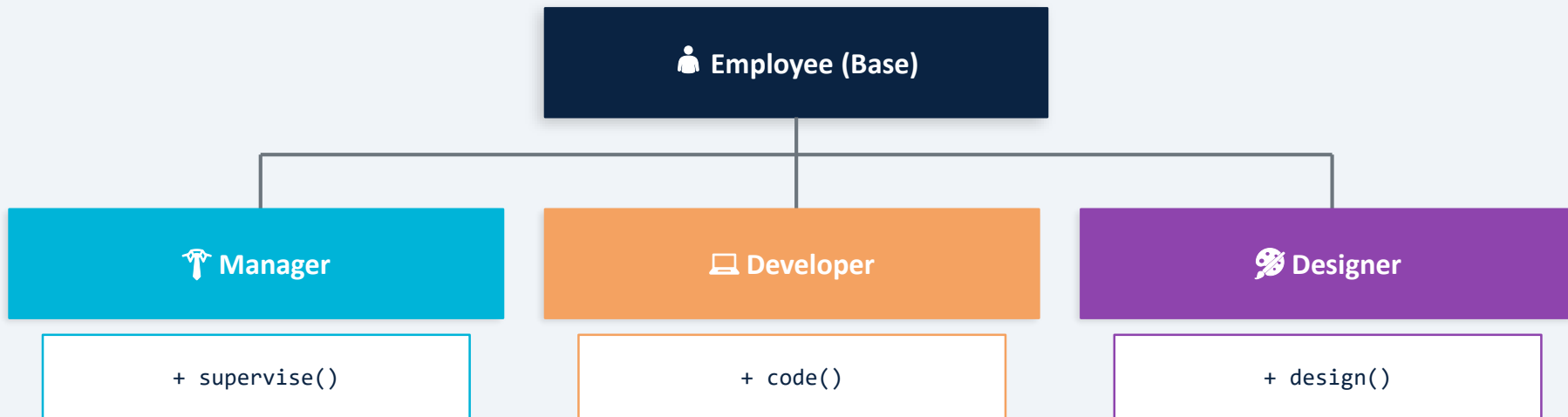
MULTILEVEL INHERITANCE – EXAMPLE [3/4]

```
class laborer : public employee    // laborer class
{
};
class foreman : public laborer    // foreman class
{
private:
    float quotas;                // percent of quotas met successfully
public:
    void getdata(){
        laborer::getdata();
        cout << "    Enter quotas: "; cin >> quotas;
    }
    void putdata() const{
        laborer::putdata();
        cout << "\n    Quotas: " << quotas;
    }
};
```

MULTILEVEL INHERITANCE – EXAMPLE [4/4]

```
int main(){
    laborer l1;
    foreman f1;
    cout << endl;
    cout << "\nEnter data for laborer 1";
    l1.getdata();
    cout << "\nEnter data for foreman 1";
    f1.getdata();
    cout << endl;
    cout << "\nData on laborer 1";
    l1.putdata();
    cout << "\nData on foreman 1";
    f1.putdata();
    cout << endl;
    return 0;
}
```

TYPE 4: HIERARCHICAL INHERITANCE — Employee System



All derived classes share common Employee base:

- **name**: string, **id**: int, **salary**: float
- `getName()`, `getSalary()`, `setDept()`
- **Manager** ADDS: `supervise()`, **teamSize**
- **Developer** ADDS: `code()`, **language**

SECTION 03

Access Specifiers

public • private • protected — Controlling Visibility in Inheritance

ACCESS SPECIFIERS IN INHERITANCE

How base class member access changes in derived class:

Base Class Member	public Inheritance	protected Inheritance	private Inheritance
public member	public	protected	private
protected member	protected	protected	private

Key Rules to Remember:

- public: Best for 'is-a' relationships. Maintains original access levels. Most commonly used.
- protected: Inherited members become protected — accessible in derived class but not outside.
- private: Everything inherited becomes private — next level derived classes can't access even protected members.
- private members are NEVER directly accessible in derived class regardless of inheritance type.

INHERITANCE TYPES — BASED ON ACCESS SPECIFIER

- Based on Access Specifier the inheritance has three types

private

protected

public

1. Public Inheritance

- Public inheritance is specified as →
- Base class private members
 - Not accessible directly
 - Still inherited
 - Manipulate through inherited member functions
- Base class public and protected members
 - Inherited with original member access
- friend functions
 - Not inherited

```
class derived_class_name : public base_class_name
{
    // data members and member functions of class A
}
```


INHERITANCE TYPES — BASED ON ACCESS SPECIFIER

- Based on Access Specifier the inheritance has three types

private

protected

public

2. Protected Inheritances

- Protected inheritance is specified as →
- Intermediate level of protection between public and private
- Protected members accessible to
 - Base class members
 - Base class friends
 - Derived class members
 - Derived class friends
- Derived-class members
 - Refer to public and protected members of base class
 - Simply use member names

```
class derived_class_name : protected base_class_name  
{  
    // data members and member functions of class A  
}
```

Protected members of a class can be accessed within the class itself and within its derived classes, but not through objects. Access to protected members must happen inside member functions of the base or derived class.

INHERITANCE TYPES — BASED ON ACCESS SPECIFIER

- Based on Access Specifier the inheritance has three types

private

protected

public

3. Private Inheritance

- Private inheritance is specified as →
- Public and protected become **private members of the derived class**
- If we do not use specifier, then by default the private inheritance is used

```
class derived_class_name : private base_class_name
{
    // data members and member functions of class A
}
```

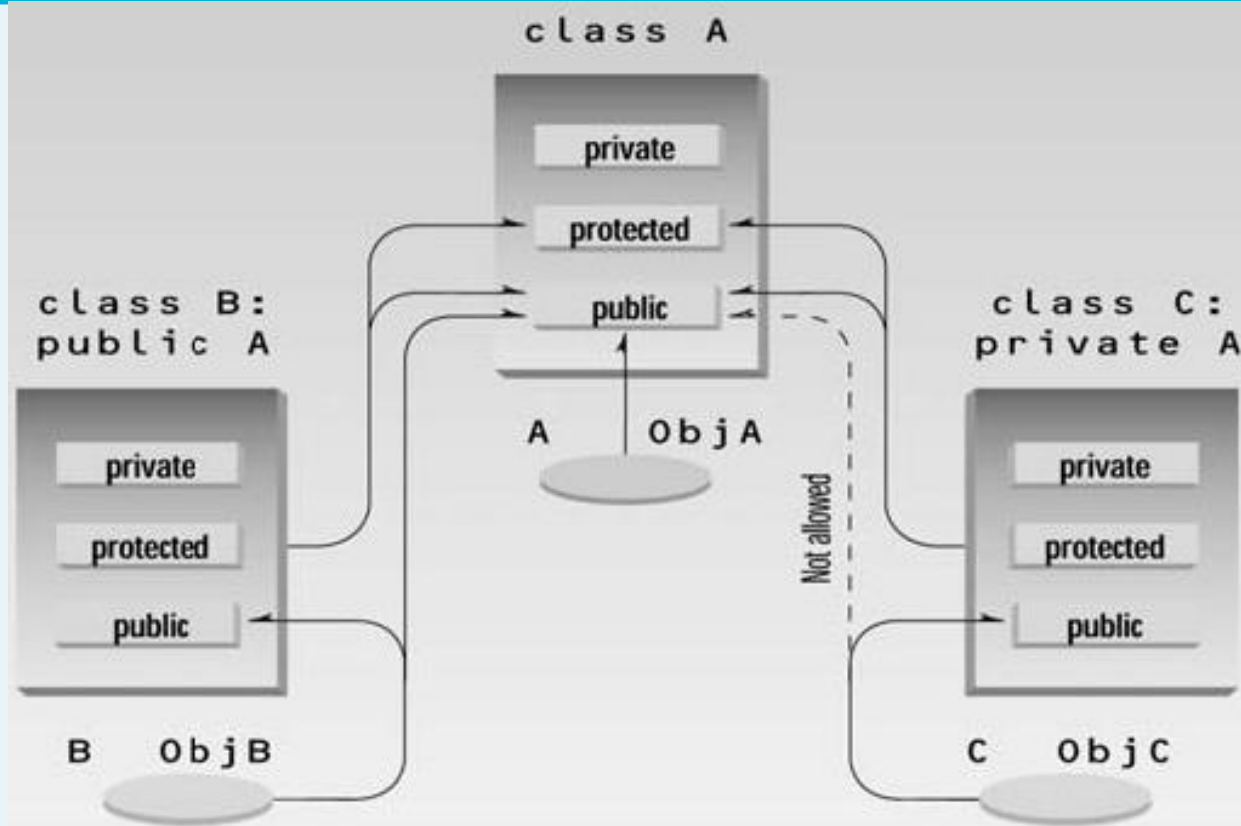
A base class's private members are never accessible directly from a derived class, but can be accessed through calls to the public and protected members of the base class

Public and Private Inheritance

class manager : public employee

- keyword **public** specifies that objects of the derived class are able to access public member functions of the base class.
- If **private** is used in place of public that means objects of the derived class cannot access public member functions of the base class, they can only access public member of their own derived class.
- Since **objects can never access private or protected members of a class**, the result is that **no member of the base class is accessible to objects of the derived class**.
- If you don't supply any access specifier when creating a class, private is assumed.

Public and Private Inheritance



Public and Private Inheritance – Example [1/2]

```
// tests publicly- and privately-derived classes
#include <iostream>
using namespace std;
class A{                                // base class
private:    int privdataA;              // (functions have the same access
protected: int protdataA;              // rules as the data shown here)
public:     int pubdataA;
};
class B : public A{                     // publicly-derived class
public:
void funct(){
    int a;
    a = privdataA;                      // error: not accessible
    a = protdataA;                      // OK
    a = pubdataA;                      // OK
}
};
```

Public and Private Inheritance – Example [2/2]

```
class C : private A{           // privately-derived class
public:
    void funct(){
        int a;
        a = privdataA;         // error: not accessible
        a = protdataA;         /* OK */
        a = pubdataA;          // OK
    } };
int main(){
    int a; B objB; a = objB.privdataA; // error: not accessible
    a = objB.protdataA;               // error: not accessible
    a = objB.pubdataA;                // OK (A public to B)
    C objC; a = objC.privdataA;       // error: not accessible
    a = objC.protdataA;               // error: not accessible
    a = objC.pubdataA; // error: not accessible (A private to C)
    return 0;
}
```

Base class member access specifier	Type of inheritance		
	Public inheritance	Protected inheritance	private inheritance
Public	public in derived class. Can be accessed directly by any non-static member functions, friend functions and non-member functions.	protected in derived class. Can be accessed directly by all non-static member functions and friend functions.	private in derived class. Can be accessed directly by all non-static member functions and friend functions.
Protected	protected in derived class. Can be accessed directly by all non-static member functions and friend functions.	protected in derived class. Can be accessed directly by all non-static member functions and friend functions.	private in derived class. Can be accessed directly by all non-static member functions and friend functions.
Private	Hidden in derived class. Can be accessed by non-static member functions and friend functions through public or protected member functions of the base class.	Hidden in derived class. Can be accessed by non-static member functions and friend functions through public or protected member functions of the base class.	Hidden in derived class. Can be accessed by non-static member functions and friend functions through public or protected member functions of the base class.

Accessing Base Class Members

Access Specifier	Accessible from own class	Accessible from derived class	Accessible from objects outside class
Public:	Yes	Yes	Yes
Protected:	Yes	Yes	No
Private;	Yes	No	No

UNDERSTANDING protected MEMBERS

protected = accessible within class AND its derived classes, but NOT outside

private

Accessible ONLY
within the class itself

protected

Accessible within class
AND all derived classes

public

Accessible from
ANYWHERE

```
class Base {
private:   int x;           // Only Base can access x
protected: int y;         // Base + Derived can access y
public:   int z;           // Anyone can access z
};

class Derived : public Base {
    void demo() {
        // x = 5;    // ✗ ERROR: private, not accessible
        y = 10;      // ✔ OK: protected accessible in derived
        z = 20;      // ✔ OK: public accessible everywhere
    }
};

int main() {
    Derived d;
    // d.y = 5;      // ✗ ERROR: protected not accessible outside
    d.z = 20;        // ✔ OK: public accessible outside
}
```